HOMECONNECT – SMART HOME AUTOMATION PLATFORM

Project Documentation

Submitted By

Vrutti Patil

Course

System Design Final Examination

---

1. Problem Statement

HomeConnect is a cloud-based Smart Home Automation Platform designed to manage and control connected IoT devices such as smart lights, thermostats, cameras, smart locks, and environmental sensors.

Modern smart homes generate a continuous stream of telemetry data, automation events, and security notifications. These events must be processed with minimal latency while maintaining security, scalability, reliability, and fault tolerance.

The platform should allow users to:

- Register and authenticate securely
- Add and manage smart devices
- Monitor real-time device status
- Create automation rules
- Receive security alerts
- Control appliances remotely
- Analyze device and sensor activity

Additionally, the architecture should support:

- IoT Gateways
- Device Authentication
- Automation Engines
- Distributed Caching
- Fault Tolerance
- Edge Device Synchronization
- Scalable Database Systems

The objective of this project is to design and implement a scalable backend system that demonstrates how modern smart home ecosystems operate.

---

2. Proposed Solution

HomeConnect provides a centralized cloud platform that enables users to control and monitor smart devices through a web-based dashboard.

The proposed solution consists of:

Frontend

A React-based web interface that provides:

- Device Management
- Automation Management
- Alert Monitoring
- Analytics Dashboard

Backend

A Flask REST API that provides:

- Authentication Services
- Device Services
- Telemetry Processing
- Automation Rule Execution
- Alert Management

Database

MySQL is used to store:

- Users
- Devices
- Automation Rules
- Telemetry Logs
- Alerts

Additional Components

To satisfy distributed system requirements, the architecture includes:

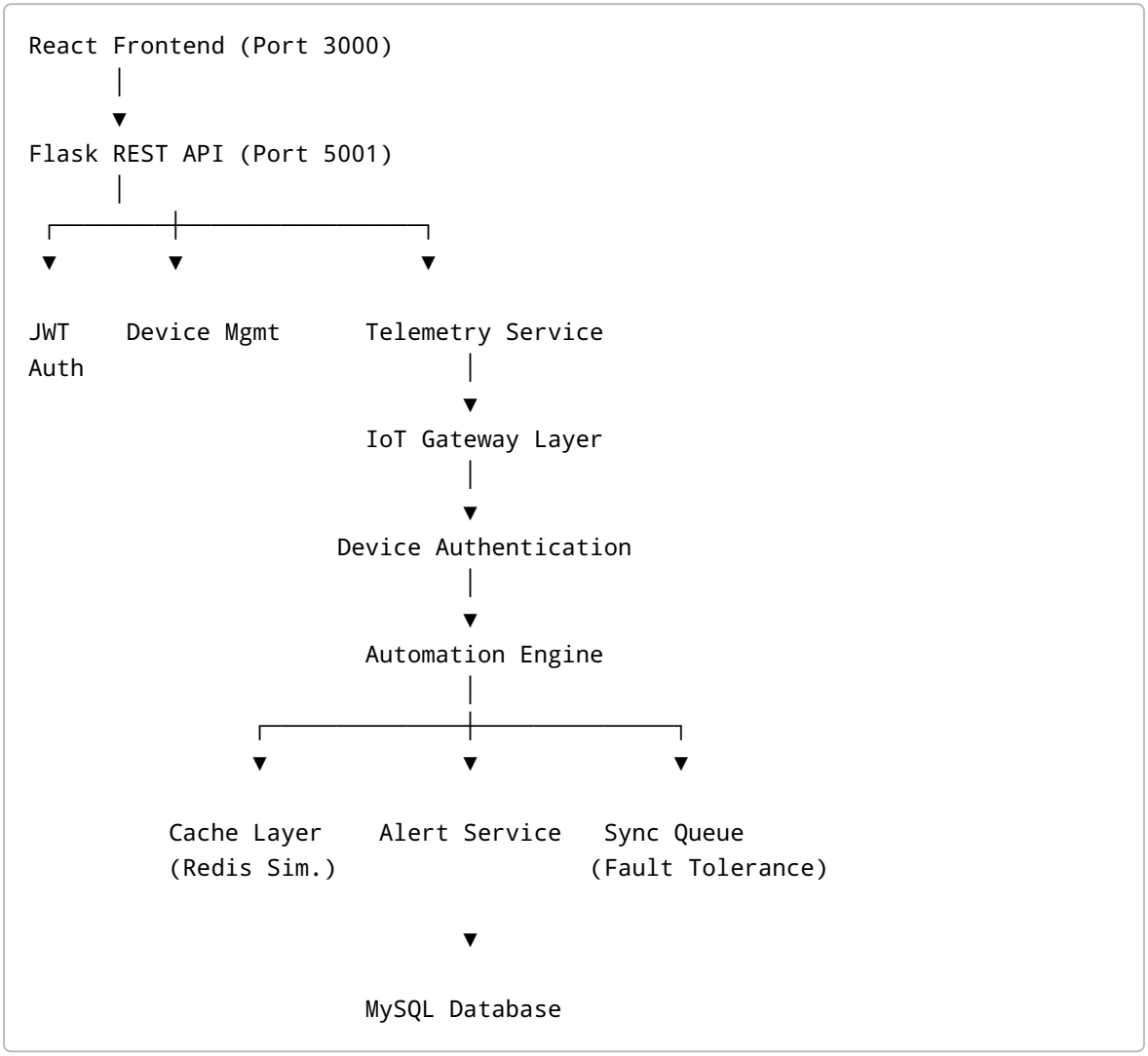
- IoT Gateway Layer
- Device Authentication Layer
- Cache Layer
- Synchronization Queue
- Health Monitoring Endpoint

This design follows principles used by enterprise smart home ecosystems such as Google Home, Amazon Alexa, and AWS IoT Core.

---

3. System Architecture

High-Level Architecture



Architecture Description

React Frontend

Provides the graphical user interface through which users interact with the platform.

Flask REST API

Acts as the central application server and processes all client requests.

JWT Authentication Layer

Secures API endpoints and verifies user identity.

Device Management Layer

Responsible for:

- Device Registration
- Device Updates
- Device Deletion
- Device Monitoring
- Remote Device Control

Telemetry Service

Receives telemetry data from smart devices and stores it in the database.

IoT Gateway Layer

Acts as an intermediate processing layer that validates telemetry data before it reaches the backend.

Device Authentication Layer

Ensures that only authorized devices can communicate with the platform.

Automation Engine

Evaluates automation rules and executes actions automatically.

Cache Layer

Stores frequently accessed device information to improve performance.

Alert Service

Generates alerts and notifications for abnormal events.

Sync Queue

Stores commands when devices are offline and synchronizes them later.

Database

Persists all application data.

---

4. Module Description

Authentication Module

Purpose

Handles user registration, login, and authentication.

Functions

- Register User
- Login User
- Generate JWT Token
- Verify JWT Token

Files

- auth.py
- auth_helper.py

Device Management Module

Purpose

Manages all smart home devices.

Functions

- Add Device
- Update Device
- Delete Device
- Control Device
- View Device Status

File

- devices.py

Telemetry Module

Purpose

Processes telemetry data generated by devices.

Functions

- Receive Sensor Data
- Store Telemetry
- Trigger Rule Evaluation

File

- telemetry.py

Automation Module

Purpose

Executes user-defined automation rules.

Functions

- Create Rule
- Update Rule
- Delete Rule
- Evaluate Rule
- Execute Action

File

- automations.py

Alert Module

Purpose

Manages system alerts and notifications.

Functions

- Generate Alerts
- Mark Alerts Read
- View Alert History

File

- alerts.py

Analytics Module

Purpose

Provides dashboard statistics and insights.

Functions

- Device Statistics
- Alert Statistics
- Telemetry Summaries

File

- analytics.py

Cache Module

Purpose

Improves performance by reducing database queries.

File

- cache_service.py

Gateway Module

Purpose

Processes telemetry before backend ingestion.

File

- gateway_service.py

Synchronization Module

Purpose

Provides fault tolerance for offline devices.

File

- sync_service.py

---

5. Database Design

Database Technology

MySQL 8.0

The database was selected because it provides:

- ACID Transactions
- Data Integrity

- Strong Relationship Support
- High Reliability

Database Tables

Users

Stores registered user information.

Field	Description
id	Primary Key
name	User Name
email	Email Address
password_hash	Encrypted Password

Devices

Stores smart device information.

Field	Description
id	Device ID
user_id	Owner
name	Device Name
type	Device Type
location	Device Location
status	Online/Offline
state	Current State

Automation Rules

Stores automation logic.

Field	Description
id	Rule ID
user_id	Owner
trigger_type	Time/Threshold

Field	Description
trigger_value	Rule Condition
action	Device Action

Telemetry Logs

Stores telemetry events.

Field	Description
id	Record ID
device_id	Device Reference
metric	Sensor Type
value	Sensor Reading
timestamp	Event Time

Alerts

Stores generated alerts.

Field	Description
id	Alert ID
device_id	Device Reference
message	Alert Message
is_read	Read Status

Entity Relationship Diagram

```
User
 │
 ├── Devices
 │
 ├── Automation Rules
 │
Device
 │
 ├── Telemetry Logs
```

```
  │
  └── Alerts
```

---

6. Technology Stack

Layer	Technology
Frontend	ReactJS
Backend	Flask
Programming Language	Python
Database	MySQL
Authentication	JWT
ORM	SQLAlchemy
API Architecture	REST
Cache	In-Memory Cache
Gateway	Simulated IoT Gateway
Version Control	Git & GitHub

---

7. Implementation Details

Authentication Implementation

User passwords are securely hashed before storage.

JWT tokens are generated after successful login and used to protect API routes.

Device Authentication

Each device is validated before telemetry data is accepted.

Unauthorized devices are rejected.

Telemetry Workflow

```
IoT Device
     │
```

```
     ▼
Telemetry Service
     │
     ▼
IoT Gateway
     │
     ▼
Device Authentication
     │
     ▼
Store Telemetry
     │
     ▼
Automation Engine
     │
     ▼
Generate Alerts
```

Automation Rule Engine

The platform supports:

Time-Based Automation

Example:

Turn Lights ON at 7:00 PM.

Threshold-Based Automation

Example:

Temperature > 35°C.

Simplified Algorithm

```
def evaluate_rule(rule, metric, value):

    if metric == rule.metric:

        if value > rule.threshold:

            execute_rule(rule)
```

When conditions evaluate to TRUE:

1. Rule executes.

2. Device state changes.
3. Alert is generated.

Cache Implementation

Frequently accessed device information is stored in cache.

Benefits:

- Faster retrieval
- Reduced database load
- Improved response time

Fault Tolerance

When a device becomes unavailable:

- Commands are stored in a synchronization queue.
- Commands are replayed after reconnection.

This prevents command loss.

Health Monitoring

Endpoint:

GET /health

Returns system status for:

- Backend
- Database
- Cache
- Gateway

---

8. Screenshots

Insert screenshots of:

Login Page

[Insert Screenshot]

Dashboard

[Insert Screenshot]

Devices Page

[Insert Screenshot]

Add Device Form

[Insert Screenshot]

Automation Rules Page

[Insert Screenshot]

Alerts Page

[Insert Screenshot]

Analytics Dashboard

[Insert Screenshot]

Health Endpoint Output

[Insert Screenshot]

Database Tables

[Insert Screenshot]

---

9. Future Scope

The platform can be extended with:

Real-Time Communication

- WebSockets
- Live Device Updates

Mobile Application

- Android Application
- iOS Application

Voice Assistants

- Amazon Alexa Integration

- Google Home Integration

Advanced Analytics

- AI-Based Automation Recommendations
- Predictive Maintenance

Cloud Scalability

- Redis Distributed Cache
- Apache Kafka Event Streaming
- MongoDB Telemetry Storage
- AWS IoT Core Integration

Enterprise Features

- Multi-Home Management
- Role-Based Access Control
- Firmware Update Management

Conclusion

HomeConnect successfully demonstrates the design and implementation of a Smart Home Automation Platform using modern cloud and distributed system principles. The system provides secure authentication, telemetry processing, automation workflows, alert generation, caching, fault tolerance, and scalability considerations. The modular architecture ensures maintainability and allows future expansion toward enterprise-scale deployments.